D0012e Lab 1

Oscar Fahlén, Filip Wåhlin, Sandra Dahlin

oscfah-1@student.ltu.se, filwih-2@student.ltu.se, asaadi-2@student.ltu.se

2023-11-12

Contents

1 Sorting algorithms **3**
1.1 Randomly selected list. 3
1.2 Sorted list. 4
1.3 Partially presorted list. 5

2 Mergesort **6**
2.1 Mergesort with bSort. 7
2.2 Mergesort with Insertion sort. 8
2.3 Mergesort comparison. 9

1 Sorting algorithms

For this lab, three different sorting algorithms where implemented. Insertions sort, bSort and Mergesort. All of these three algorithms has a time complexity. The time complexity in the worst case can be seen in the table below.

Algorithm	Big-O value
Insertion sort	$O(n^2)$
bSort	$O(n^2)$
Mergesort	$O(nLog(n))$

Table 1: The Big-O values for n number of operations in the worst case.

1.1 Randomly selected list.

The different values of O results in a vastly different execution time for each algorithm.

Even though Insertion sort and bSort have similar O-values in the worst case

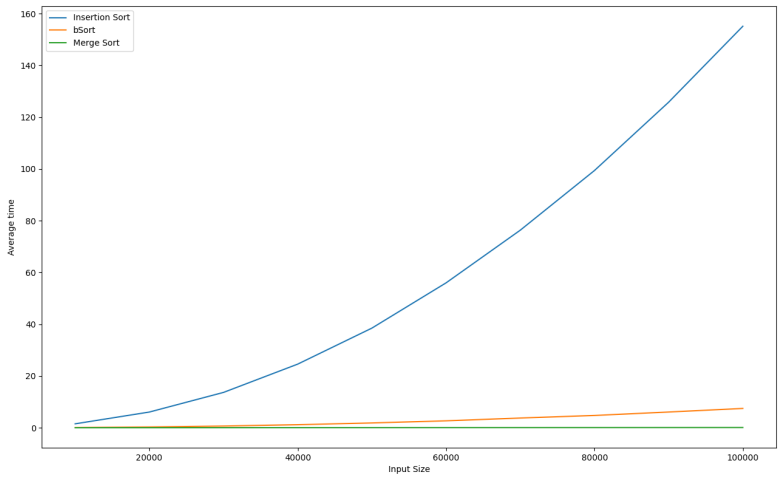


Figure 1: Execution time for each implemented algorithm. During this simulation the input elements where randomly selected. For every data point of each algorithm, the time is the average over 10 iterations.

they differ immensely in their execution time. This is because bSort provides a much more efficient search method than the linear search utilized by the normal insertion sort. This difference in execution time can be seen in figure 1 where each algorithm was used to sort random lists with a length varying from ten to

a hundred thousand elements. The fastest algorithm by far of the three once tested was Mergesort.

1.2 Sorted list.

After testing the algorithms on randomly generated lists the same test was performed using lists sorted in increasing order. Out of the three algorithms tested, bSort gave the worst execution time. This is because of the fact that the binary search used by bSort performs much worse in the best case than the linear search of the normal insertion sort does, particularly for large inputs.

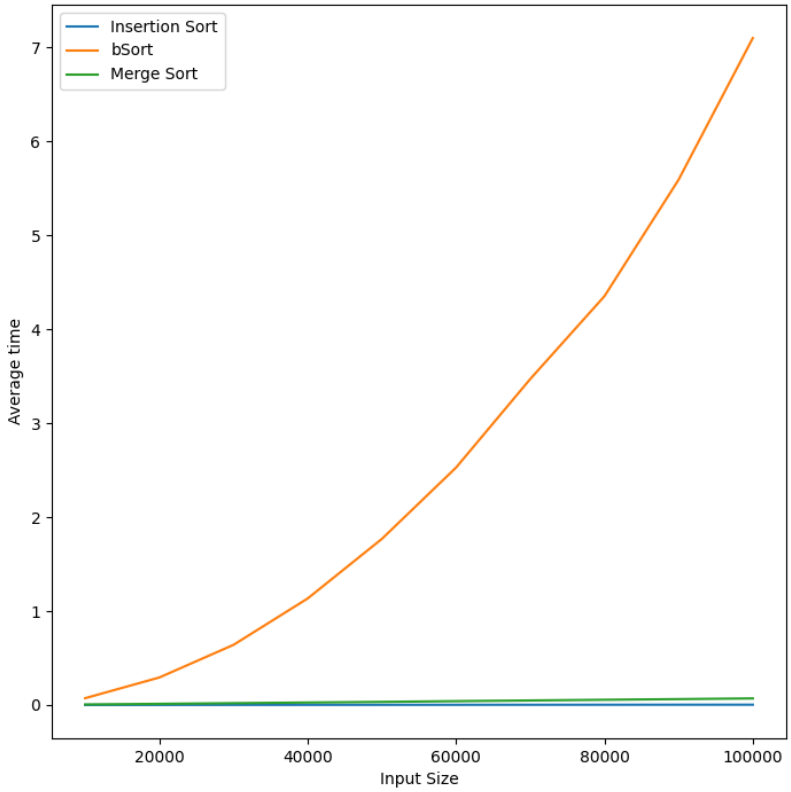


Figure 2: Execution time for each implemented algorithm. The execution time for each datapoint is taken as the average time of ten iterations.

1.3 Partially presorted list.

Partially presorted list refers to a list where the first and last 3/8th of the list is presorted and middle is randomly generated containing values bigger then the values of the elements in the first 3/8th and smaller then the values of the elements of the last 3/8th of the list. The result shows similar patterns to the randomly generated list. That insertion sort performances this poorly compared to the other algorithms while the size input increases, is nothing unexpected. In the same test when all list was fully unsorted the performances compared to the other algorithms was proportionally as bad. A partially presorted list with insertion sort still needs to compare and move most elements back and forth. In this case the presorted elements make only a slight difference in running time compared to the other algorithms. While bSort searching for the index where the element is supposed to be placed more efficiently, makes it much quicker in running time than insertion sort. Between randomly and partially sorted list there is a small difference, but when it comes to a fully sorted list its performs much worse. Since the algorithm still picks out and insert the element in the list which takes time. Mergesort time wise is much faster than both but Insertionsort and bSort but has the trade off of using additional memory.

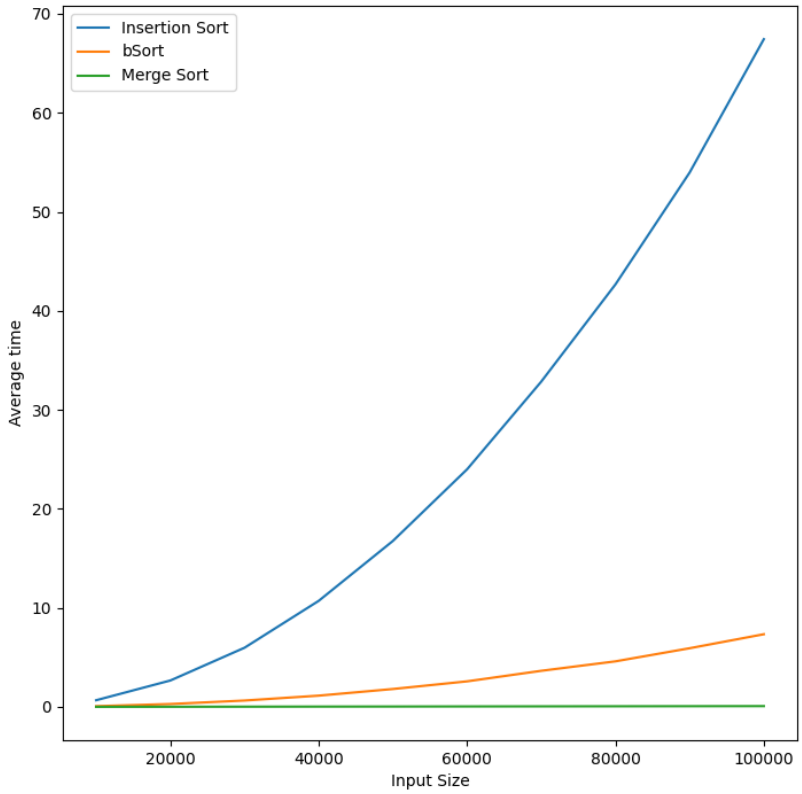


Figure 3: Execution time for each implemented algorithm. Each point show the average running time for 10 different lists. Each list goes from ten to one hundred thousand randomly created integers, where three-quarters of the list is already presorted in raising values

2 Mergesort

The optimal K-value was experimentally calculated by running a simulation over different values for K, these values ran between 1 to 200 with an increment of 1 for each run. The runs included one hundred thousand randomly selected integers. The shown time value for each value of k was the average runtime over 10 iterations. This was repeated for both Mergesort running with bSort and Insertionsort. The runtime for different values of k was not checked above 200. Because the results had started to become worse than in previous results. This makes intuitive sense as previous results have shown both Insertionsort

and bSort to perform worse than Mergesort for larger input sizes.

2.1 Mergesort with bSort.

When running Mergesort with bSort included, the optimal K value is approximately 50, somewhat dependent on run to run variance and input length. When observing the related plot a clear stair-stepping pattern can be seen. This pattern is a result of how this implementation of the algorithm ensures that each of the base case list has a length of exactly K.

This is done using the following procedure. First the algorithm checks if the length of the list is $k \cdot 2^i$ if so the list is split exactly in half, like in the standard merge sort implementation. If that is not the case the list is split into two parts, one that is $k \cdot 2^i$ in length and one that is the remaining length of the original list. For each call of the algorithm i is the largest possible power of 2 such that $k \cdot 2^i \leq n$ where n is the length of the list.

The best K value for the algorithm is the value where the two parts of the list are roughly even in length. The worse K values for the algorithm are where the two parts of the list are very uneven in length. The uneven division of the lists result in the algorithm having to recursively sort one list that is much longer than if the split would have been even which takes longer time. This interestingly means that the optimal K value varies slightly with the length of the input.

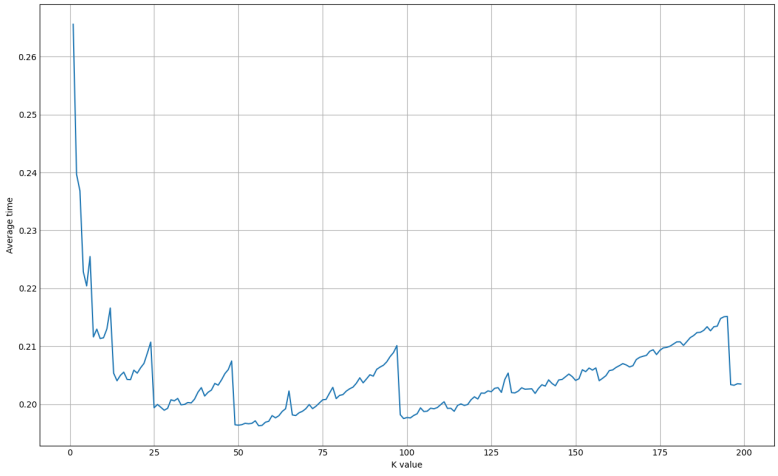


Figure 4: Execution time for Mergesort given different values for K.

2.2 Mergesort with Insertion sort.

Insertion sort is most efficient when handling list of a shorter length. So is also the case for Mergesort running with insertion sort, the optimal K value is therefore as expected lower than when running using bSort at around 10, also somewhat dependent on run to run variance as well as input length. For an explanation of the connection between input length and optimal K value as well as the slight stair-stepping that can be observed in the plot see section 2.1. The reason that the stair-stepping is less pronounced when using Insertion sort is because the runtime of insertion sort overshadows the effect uneven division of the lists has on the total running time of the algorithm.

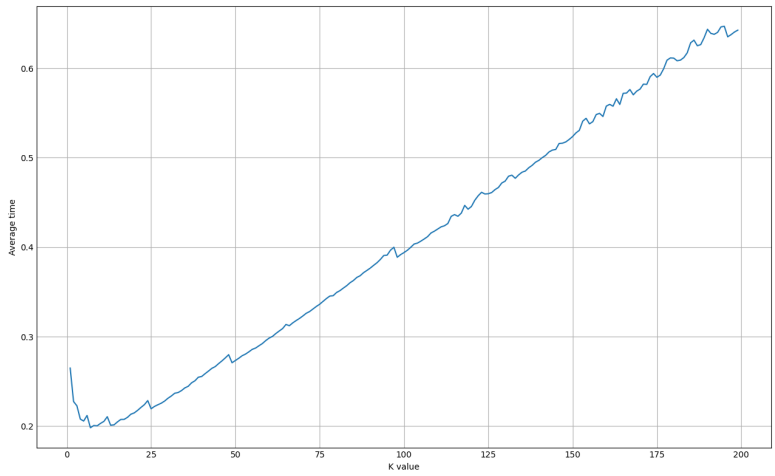


Figure 5: Graph for extracting the optimal K value for merge sort with Insertion sort included.

2.3 Mergesort comparison.

During this simulation the different types of Mergesort where tested against each other, all with their respective optimal K value. Standard Mergesort were in all cases worse then the other mergesorts. Both Mergesort running with insertion and bSort gave very similar results only deviating by a small margin from each other.

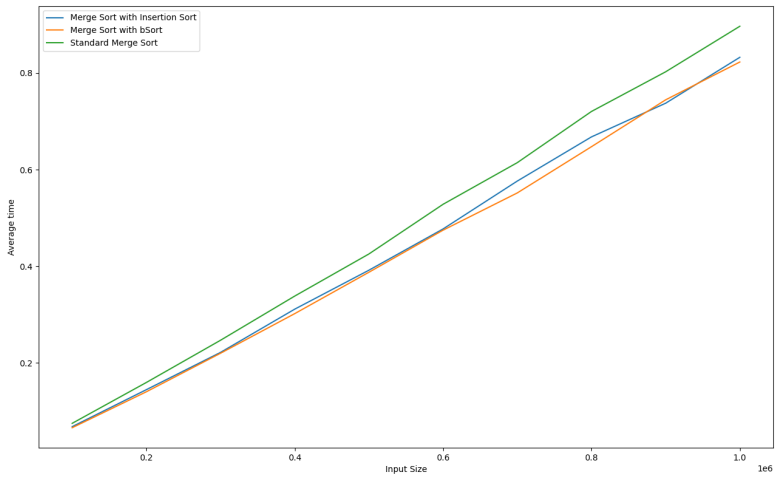


Figure 6: Average time for different variants of merge sort.